

pandas 데이터 파악과 조작

분석할 데이터를 수집(확보)하면 데이터의 특징을 파악하고 다루기 쉽게 변형하는 작업을 수행해야 한다

#2. 데이터 조작(가공)

- 데이터 개수 세기 : `count()`, `value_counts()`, `unique()`
- 데이터 정렬 : `sort_values()`, `sort_index()`
- 데이터 순위 : `rank()`
- 데이터 집계 : 합계(`sum()`), 평균(`mean()`), 최대(`max()`), 최소(`min()`), 분산, 표준편차, 중위수, 상관계수
- 데이터 삭제 : `drop(axis=0|1)`
- 결측치 파악 : `isna()`, `isnull()` + `sum()`
- 결측치 처리 : `dropna(axis=0|1, subset, inplace)`, `fillna(value=, limit=)`
- 데이터 변경 :
 - 자료형 변경 : `astype()`
 - 수치형 데이터를 범주형 데이터로 변경 :
 - 구간을 지정하여 범주화 : `cut(data, bins, labels)`
 - 동일한 개수를 갖도록 범주화 : `qcut(data, bins_num, labels)`
- 행/열에 동일한 함수 적용 : `apply()`, `applymap()`, `map()`

```
In [1]: import numpy as np
import pandas as pd
print(f'Numpy:{np.__version__}, Pandas:{pd.__version__}')
```

Numpy:2.5.2, Pandas:3.0.5

```
In [2]: from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
```

4. 데이터 삭제

- **Series.drop**(labels=None, axis=0, index=None, columns=None, level=None, inplace=False, errors='raise')
- **DataFrame.drop**(labels=None, axis=0, index=None, columns=None, level=None, inplace=False, errors='raise')

1) 데이터프레임의 행 삭제

- `df.drop(labels='행이름', axis=0, inplace=False)` : 행삭제
- `df.drop(index=['행이름1', '행이름2', ...])`
- 행삭제 후 데이터프레임으로 결과 반환
- 원본에 반영되지 않으므로 원본수정하려면 저장해야 함
- 객체변수로 저장하거나, 매개변수 `inplace`를 `True`로 지정

```
In [3]: np.random.seed(1)

df1 = pd.DataFrame(np.random.randint(10, size=(4,8)))
df1
```

```
Out[3]:
```

	0	1	2	3	4	5	6	7
0	5	8	9	5	0	0	1	7
1	6	9	2	4	5	2	4	2
2	4	7	7	9	1	7	0	6
3	9	9	7	6	9	1	0	1

```
In [4]: df1['Total'] = df1.sum(axis='columns')
df1.loc['Sum'] = df1.sum()
df1
```

```
Out[4]:
```

	0	1	2	3	4	5	6	7	Total
0	5	8	9	5	0	0	1	7	35
1	6	9	2	4	5	2	4	2	34
2	4	7	7	9	1	7	0	6	41
3	9	9	7	6	9	1	0	1	42
Sum	24	33	25	24	15	10	5	16	152

```
In [5]: df1.columns = [f'col{i}' for i in range(1,9)] + ['Total']
df1
```

```
Out[5]:
```

	col1	col2	col3	col4	col5	col6	col7	col8	Total
0	5	8	9	5	0	0	1	7	35
1	6	9	2	4	5	2	4	2	34
2	4	7	7	9	1	7	0	6	41
3	9	9	7	6	9	1	0	1	42
Sum	24	33	25	24	15	10	5	16	152

```
In [6]: df1 = df1.rename(index={0:'idx1', 1:'idx2', 2:'idx3', 3:'idx4'})
df1
```

```
Out[6]:
```

	col1	col2	col3	col4	col5	col6	col7	col8	Total
idx1	5	8	9	5	0	0	1	7	35
idx2	6	9	2	4	5	2	4	2	34
idx3	4	7	7	9	1	7	0	6	41
idx4	9	9	7	6	9	1	0	1	42
Sum	24	33	25	24	15	10	5	16	152

- df1의 Sum 행 삭제

```
In [7]: df1.drop(index='Sum', columns='Total')
df1
```

```
Out[7]:
```

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	5	8	9	5	0	0	1	7
idx2	6	9	2	4	5	2	4	2
idx3	4	7	7	9	1	7	0	6
idx4	9	9	7	6	9	1	0	1

```
Out[7]:
```

	col1	col2	col3	col4	col5	col6	col7	col8	Total
idx1	5	8	9	5	0	0	1	7	35
idx2	6	9	2	4	5	2	4	2	34
idx3	4	7	7	9	1	7	0	6	41
idx4	9	9	7	6	9	1	0	1	42
Sum	24	33	25	24	15	10	5	16	152

```
In [8]: df1.drop(axis=0, labels='Sum')
```

```
Out[8]:
```

	col1	col2	col3	col4	col5	col6	col7	col8	Total
idx1	5	8	9	5	0	0	1	7	35
idx2	6	9	2	4	5	2	4	2	34
idx3	4	7	7	9	1	7	0	6	41
idx4	9	9	7	6	9	1	0	1	42

2) 데이터프레임의 열 삭제

- `df.drop(labels='열이름', axis=1, inplace=False)` : 열 삭제
- `df.drop(columns=['열이름1','열이름2',...])`
- 행삭제 후 데이터프레임으로 결과 반환

```
In [9]: df1.drop(axis=1, labels='Total')
```

```
Out[9]:
```

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	5	8	9	5	0	0	1	7
idx2	6	9	2	4	5	2	4	2
idx3	4	7	7	9	1	7	0	6
idx4	9	9	7	6	9	1	0	1
Sum	24	33	25	24	15	10	5	16

```
In [10]: df1.drop(columns='Total')
```

Out[10]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	5	8	9	5	0	0	1	7
idx2	6	9	2	4	5	2	4	2
idx3	4	7	7	9	1	7	0	6
idx4	9	9	7	6	9	1	0	1
Sum	24	33	25	24	15	10	5	16

In [11]: `df1.drop(columns=['col1', 'col3'])`

Out[11]:

	col2	col4	col5	col6	col7	col8	Total
idx1	8	5	0	0	1	7	35
idx2	9	4	5	2	4	2	34
idx3	7	9	1	7	0	6	41
idx4	9	6	9	1	0	1	42
Sum	33	24	15	10	5	16	152

5. 결측치 처리

: NaN 값 처리 함수

- **df.dropna(axis=0 또는 1, inplace=False)**
 - NaN값이 있는 열 또는 행을 삭제
 - 원본 반영되지 않음
 - inplace=True : 원본 데이터프레임 변경
- **df.fillna(0, inplace=False)**
 - NaN값을 정해진 숫자로 채움
 - 원본 반영 되지 않음
- **df.ffill(), df.bfill()**

결측치 적용

In [12]: `np.random.seed(10)`
`df2 = pd.DataFrame(data=np.random.randint(10, size=(4,8)),`
`columns = [f'col{i}' for i in range(1,9)],`
`index = [f'idx{i}' for i in range(1,5)])`
`df2`

Out[12]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	9	4	0	1	9	0	1	8
idx2	9	0	8	6	4	3	0	4
idx3	6	8	1	8	4	1	3	6
idx4	5	3	9	6	9	1	9	4

```
In [13]: df2.iloc[0,0] = np.nan
df2.iloc[2,3] = np.nan
df2.iloc[1,2] = np.nan
df2
```

```
Out[13]:
```

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9	0	1	8
idx2	9.0	0	NaN	6.0	4	3	0	4
idx3	6.0	8	1.0	NaN	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

```
In [14]: df2.info()

<class 'pandas.DataFrame'>
Index: 4 entries, idx1 to idx4
Data columns (total 8 columns):
#   Column  Non-Null Count  Dtype  
---  -
0   col1    3 non-null      float64
1   col2    4 non-null      int32   
2   col3    3 non-null      float64
3   col4    3 non-null      float64
4   col5    4 non-null      int32   
5   col6    4 non-null      int32   
6   col7    4 non-null      int32   
7   col8    4 non-null      int32   
dtypes: float64(3), int32(5)
memory usage: 208.0+ bytes
```

0) 결측치 확인 : isna(), isnull()

```
In [15]: df2.isna()
df2.isnull()
```

```
Out[15]:
```

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	True	False	False	False	False	False	False	False
idx2	False	False	True	False	False	False	False	False
idx3	False	False	False	True	False	False	False	False
idx4	False	False	False	False	False	False	False	False

```
Out[15]:
```

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	True	False	False	False	False	False	False	False
idx2	False	False	True	False	False	False	False	False
idx3	False	False	False	True	False	False	False	False
idx4	False	False	False	False	False	False	False	False

```
In [16]: df2.isna().sum()
```

```
Out[16]: col1      1
          col2      0
          col3      1
          col4      1
          col5      0
          col6      0
          col7      0
          col8      0
          dtype: int64
```

```
In [17]: # 결측치가 있는 컬럼이름 추출하기
df2.isna().sum() > 0
df2.columns[df2.isna().sum() > 0]
```

```
Out[17]: col1      True
          col2     False
          col3      True
          col4      True
          col5     False
          col6     False
          col7     False
          col8     False
          dtype: bool
```

```
Out[17]: Index(['col1', 'col3', 'col4'], dtype='str')
```

```
In [19]: # 타이타닉 데이터에서 결측치가 있는 컬럼 추출하기
df_tit = pd.read_csv('data/train.csv')
df_tit.columns[df_tit.isnull().sum()>0]
```

```
Out[19]: Index(['Age', 'Cabin', 'Embarked'], dtype='str')
```

결측치 처리 방법

- 삭제 : 결측치가 있는 행이나 열을 삭제
 - 결측치 포함 비율에 따라 삭제할지 결정: 의미성 여부
 - 무조건 삭제하면 안됨
- 대체(imputation) : 다른 값으로 대체
 - 평균값으로 대체 : 해당 컬럼의 평균값, 값이 하양조정될 가능성이 많음
 - 핫덱(hot-deck) 대체 : 결측치가 있는 관측값(행기준)들과 유사한 관측값들이 가지는 값으로 랜덤하게 배정
 - 알고리즘 활용 : 회귀분석, 최근접이웃법(K-Nearest Neighbors:KNN),... (보간법)
 - 앞 또는 뒤의 값으로 대체 : 시계열데이터와 같이 앞 뒤 데이터가 유사하거나 동일한 값으로 구성된 경우

1) 결측치 포함 행 삭제 : dropna()

- dropna(axis=0) 또는 dropna(axis='index')

```
In [20]: df2
df2.dropna()
```

Out[20]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9	0	1	8
idx2	9.0	0	NaN	6.0	4	3	0	4
idx3	6.0	8	1.0	NaN	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

Out[20]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx4	5.0	3	9.0	6.0	9	1	9	4

2) 결측치 포함 열 삭제 : dropna(axis=1)

- dropna(axis=1) 또는 dropna(axis='columns')

In [21]:

```
df2
df2.dropna(axis=1)
```

Out[21]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9	0	1	8
idx2	9.0	0	NaN	6.0	4	3	0	4
idx3	6.0	8	1.0	NaN	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

Out[21]:

	col2	col5	col6	col7	col8
idx1	4	9	0	1	8
idx2	0	4	3	0	4
idx3	8	4	1	3	6
idx4	3	9	1	9	4

In [23]:

```
df2
df2.dropna(axis='columns', subset=['idx1','idx3','idx4'])
```

Out[23]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9	0	1	8
idx2	9.0	0	NaN	6.0	4	3	0	4
idx3	6.0	8	1.0	NaN	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

Out[23]:

	col2	col3	col5	col6	col7	col8
idx1	4	0.0	9	0	1	8
idx2	0	NaN	4	3	0	4
idx3	8	1.0	4	1	3	6
idx4	3	9.0	9	1	9	4

```
In [24]: df2
df2.dropna(subset=['col1'])
```

Out[24]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9	0	1	8
idx2	9.0	0	NaN	6.0	4	3	0	4
idx3	6.0	8	1.0	NaN	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

Out[24]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx2	9.0	0	NaN	6.0	4	3	0	4
idx3	6.0	8	1.0	NaN	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

```
In [36]: df2
df2.dropna(how='all') # 한 행의 모든 데이터가 결측치인 경우 해당 행 삭제
```

Out[36]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9	0	1	8
idx2	9.0	0	NaN	6.0	4	3	0	4
idx3	6.0	8	1.0	NaN	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

Out[36]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9	0	1	8
idx2	9.0	0	NaN	6.0	4	3	0	4
idx3	6.0	8	1.0	NaN	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

```
In [28]: df3 = pd.DataFrame(np.random.randint(7,size=(4,4)),
                           columns=['A','B','C','D'],
                           index=['a','b','c','d'])

df3
for i in range(4):
    df3.iloc[0,i] = np.nan
df3.iloc[2,3] = np.nan
df3
```


Out[28]:

	A	B	C	D
a	3	2	5	5
b	6	6	0	3
c	4	2	0	1
d	2	0	6	0

Out[28]:

	A	B	C	D
a	NaN	NaN	NaN	NaN
b	6.0	6.0	0.0	3.0
c	4.0	2.0	0.0	NaN
d	2.0	0.0	6.0	0.0

In [29]: df3.dropna()

Out[29]:

	A	B	C	D
b	6.0	6.0	0.0	3.0
d	2.0	0.0	6.0	0.0

In [30]: df3.dropna(axis=1)

Out[30]:

a
b
c
d

In [31]: df3.dropna(how='all')

Out[31]:

	A	B	C	D
b	6.0	6.0	0.0	3.0
c	4.0	2.0	0.0	NaN
d	2.0	0.0	6.0	0.0

In [33]: df3
for i in range(4):
 df3.iloc[i,0] = np.nan
df3

Out[33]:

	A	B	C	D
a	NaN	NaN	NaN	NaN
b	6.0	6.0	0.0	3.0
c	4.0	2.0	0.0	NaN
d	2.0	0.0	6.0	0.0

Out[33]:

	A	B	C	D
a	NaN	NaN	NaN	NaN
b	NaN	6.0	0.0	3.0
c	NaN	2.0	0.0	NaN
d	NaN	0.0	6.0	0.0

In [34]: `df3.dropna(axis=1)`

Out[34]:

a
b
c
d

In [35]: `df3.dropna(axis=1, how='all')`

Out[35]:

	B	C	D
a	NaN	NaN	NaN
b	6.0	0.0	3.0
c	2.0	0.0	NaN
d	0.0	6.0	0.0

3) 결측치를 다른 값으로 대체 : fillna() 함수

- 함수 형식:

```
DataFrame.fillna(value, *, axis=None, inplace=False, limit=None)
```

In [37]: `df2`

Out[37]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9	0	1	8
idx2	9.0	0	NaN	6.0	4	3	0	4
idx3	6.0	8	1.0	NaN	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

결측치를 0으로 변경

In [38]: `df2.fillna(-999)`

Out[38]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	-999.0	4	0.0	1.0	9	0	1	8
idx2	9.0	0	-999.0	6.0	4	3	0	4
idx3	6.0	8	1.0	-999.0	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

In [40]: df2.fillna(1000, axis=1)

Out[40]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	1000.0	4.0	0.0	1.0	9.0	0.0	1.0	8.0
idx2	9.0	0.0	1000.0	6.0	4.0	3.0	0.0	4.0
idx3	6.0	8.0	1.0	1000.0	4.0	1.0	3.0	6.0
idx4	5.0	3.0	9.0	6.0	9.0	1.0	9.0	4.0

In [41]: df2.fillna(0)

Out[41]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	0.0	4	0.0	1.0	9	0	1	8
idx2	9.0	0	0.0	6.0	4	3	0	4
idx3	6.0	8	1.0	0.0	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

결측치를 1로 변경

In [42]: df2.fillna(1)

Out[42]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	1.0	4	0.0	1.0	9	0	1	8
idx2	9.0	0	1.0	6.0	4	3	0	4
idx3	6.0	8	1.0	1.0	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

- 결측치를 처리하려는 컬럼에 대해서만 대체하려면?

In [45]: df2
df2['col4'] = df2['col4'].fillna(0)
df2

Out[45]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9	0	1	8
idx2	9.0	0	NaN	6.0	4	3	0	4
idx3	6.0	8	1.0	NaN	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

Out[45]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9	0	1	8
idx2	9.0	0	NaN	6.0	4	3	0	4
idx3	6.0	8	1.0	0.0	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

- 지정한 행에 대해서만 결측치를 처리하는 경우는?

In [47]: `df2.loc['idx1'] = df2.loc['idx1'].fillna(-1000)`
df2

Out[47]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	-1000.0	4	0.0	1.0	9	0	1	8
idx2	9.0	0	NaN	6.0	4	3	0	4
idx3	6.0	8	1.0	0.0	4	1	3	6
idx4	5.0	3	9.0	6.0	9	1	9	4

결측치를 평균으로 변경

In [48]: `df2.iloc[0,0], df2.iloc[3,4] = np.nan, np.nan`
df2

Out[48]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9.0	0	1	8
idx2	9.0	0	NaN	6.0	4.0	3	0	4
idx3	6.0	8	1.0	0.0	4.0	1	3	6
idx4	5.0	3	9.0	6.0	NaN	1	9	4

In [49]: `df2['col5'].mean()`
`df2['col5'].fillna(df2['col5'].mean())`

Out[49]: np.float64(5.666666666666667)

Out[49]:

idx1	9.000000
idx2	4.000000
idx3	4.000000
idx4	5.666667

Name: col5, dtype: float64

결측치를 앞(뒤)의 값으로 변경

`DataFrame.ffill(*, axis=None, inplace=False, limit=None, limit_area=None)`
`DataFrame.bfill(*, axis=None, inplace=False, limit=None, limit_area=None)`

In [51]: `df2`
`df2.ffill()` # 앞의 값으로 대체
`df2.bfill()` # 뒤의 값으로 대체

Out[51]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9.0	0	1	8
idx2	9.0	0	NaN	6.0	4.0	3	0	4
idx3	6.0	8	1.0	0.0	4.0	1	3	6
idx4	5.0	3	9.0	6.0	NaN	1	9	4

Out[51]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9.0	0	1	8
idx2	9.0	0	0.0	6.0	4.0	3	0	4
idx3	6.0	8	1.0	0.0	4.0	1	3	6
idx4	5.0	3	9.0	6.0	4.0	1	9	4

Out[51]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	9.0	4	0.0	1.0	9.0	0	1	8
idx2	9.0	0	1.0	6.0	4.0	3	0	4
idx3	6.0	8	1.0	0.0	4.0	1	3	6
idx4	5.0	3	9.0	6.0	NaN	1	9	4

In [52]:

```
df2
df2.ffill(axis=1)
df2.bfill(axis=1)
```

Out[52]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9.0	0	1	8
idx2	9.0	0	NaN	6.0	4.0	3	0	4
idx3	6.0	8	1.0	0.0	4.0	1	3	6
idx4	5.0	3	9.0	6.0	NaN	1	9	4

Out[52]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4.0	0.0	1.0	9.0	0.0	1.0	8.0
idx2	9.0	0.0	0.0	6.0	4.0	3.0	0.0	4.0
idx3	6.0	8.0	1.0	0.0	4.0	1.0	3.0	6.0
idx4	5.0	3.0	9.0	6.0	6.0	1.0	9.0	4.0

Out[52]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	4.0	4.0	0.0	1.0	9.0	0.0	1.0	8.0
idx2	9.0	0.0	6.0	6.0	4.0	3.0	0.0	4.0
idx3	6.0	8.0	1.0	0.0	4.0	1.0	3.0	6.0
idx4	5.0	3.0	9.0	6.0	1.0	1.0	9.0	4.0

In [53]:

```
df3
```

Out[53]:

	A	B	C	D
a	NaN	NaN	NaN	NaN
b	NaN	6.0	0.0	3.0
c	NaN	2.0	0.0	NaN
d	NaN	0.0	6.0	0.0

```
In [54]: df3.bfill()  
df3.ffill()  
df3.bfill(axis=1)  
df3.ffill(axis=1)
```

Out[54]:

	A	B	C	D
a	NaN	6.0	0.0	3.0
b	NaN	6.0	0.0	3.0
c	NaN	2.0	0.0	0.0
d	NaN	0.0	6.0	0.0

Out[54]:

	A	B	C	D
a	NaN	NaN	NaN	NaN
b	NaN	6.0	0.0	3.0
c	NaN	2.0	0.0	3.0
d	NaN	0.0	6.0	0.0

Out[54]:

	A	B	C	D
a	NaN	NaN	NaN	NaN
b	6.0	6.0	0.0	3.0
c	2.0	2.0	0.0	NaN
d	0.0	0.0	6.0	0.0

Out[54]:

	A	B	C	D
a	NaN	NaN	NaN	NaN
b	NaN	6.0	0.0	3.0
c	NaN	2.0	0.0	0.0
d	NaN	0.0	6.0	0.0

컬럼별 결측치 다르게 대체

- 컬럼마다 결측치를 대체하는 값을 다르게 처리

```
df.fillna(value={'컬럼명1':값1, '컬럼명2':값2,...,'컬럼명n':값n})
```

```
In [60]: df2  
df2.fillna(value={'col5':0, 'col1':-999, 'col3':df2['col3'].mean()}, inplace=True)
```

Out[60]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	NaN	4	0.0	1.0	9.0	0	1	8
idx2	9.0	0	NaN	6.0	4.0	3	0	4
idx3	6.0	8	1.0	0.0	4.0	1	3	6
idx4	5.0	3	9.0	6.0	NaN	1	9	4

Out[60]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	-999.0	4	0.000000	1.0	9.0	0	1	8
idx2	9.0	0	3.333333	6.0	4.0	3	0	4
idx3	6.0	8	1.000000	0.0	4.0	1	3	6
idx4	5.0	3	9.000000	6.0	0.0	1	9	4

Out[60]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	-999.0	4	0.000000	1.0	9.0	0	1	8
idx2	9.0	0	3.333333	6.0	4.0	3	0	4
idx3	6.0	8	1.000000	0.0	4.0	1	3	6
idx4	5.0	3	9.000000	6.0	0.0	1	9	4

6. 데이터의 변경

1) 데이터 자료형(dtype) 변경

astype(데이터형)

- astype(dtype)
- astype({컬럼:dtype,...})-

In [61]: df2

Out[61]:

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	-999.0	4	0.000000	1.0	9.0	0	1	8
idx2	9.0	0	3.333333	6.0	4.0	3	0	4
idx3	6.0	8	1.000000	0.0	4.0	1	3	6
idx4	5.0	3	9.000000	6.0	0.0	1	9	4

In [62]: df2.info()

```
<class 'pandas.DataFrame'>
Index: 4 entries, idx1 to idx4
Data columns (total 8 columns):
#   Column  Non-Null Count  Dtype
---  -
0   col1    4 non-null      float64
1   col2    4 non-null      int32
2   col3    4 non-null      float64
3   col4    4 non-null      float64
4   col5    4 non-null      float64
5   col6    4 non-null      int32
6   col7    4 non-null      int32
7   col8    4 non-null      int32
dtypes: float64(4), int32(4)
memory usage: 396.0+ bytes
```

```
In [63]: df2.dtypes
```

```
Out[63]: col1    float64
col2     int32
col3    float64
col4    float64
col5    float64
col6     int32
col7     int32
col8     int32
dtype: object
```

데이터를 정수형으로 변경 : astype(int)

```
In [65]: df2.astype(int)
```

```
Out[65]:
```

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	-999	4	0	1	9	0	1	8
idx2	9	0	3	6	4	3	0	4
idx3	6	8	1	0	4	1	3	6
idx4	5	3	9	6	0	1	9	4

데이터를 실수형으로 변경 : astype(float)

```
In [ ]: df.drop_duplicates(subset=['c1','c3'])
```

컬럼별 데이터 유형 변경

```
In [66]: df2
```

```
Out[66]:
```

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	-999.0	4	0.000000	1.0	9.0	0	1	8
idx2	9.0	0	3.333333	6.0	4.0	3	0	4
idx3	6.0	8	1.000000	0.0	4.0	1	3	6
idx4	5.0	3	9.000000	6.0	0.0	1	9	4

```
In [67]: df2_ = df2.astype({'col1':'int32', 'col3':'float32', 'col4':int, 'col5':int})
df2_
df2_.info()
```



```
Out[67]:
```

	col1	col2	col3	col4	col5	col6	col7	col8
idx1	-999	4	0.000000	1	9	0	1	8
idx2	9	0	3.333333	6	4	3	0	4
idx3	6	8	1.000000	0	4	1	3	6
idx4	5	3	9.000000	6	0	1	9	4

```
<class 'pandas.DataFrame'>
Index: 4 entries, idx1 to idx4
Data columns (total 8 columns):
#   Column  Non-Null Count  Dtype
---  -
0   col1    4 non-null      int32
1   col2    4 non-null      int32
2   col3    4 non-null      float32
3   col4    4 non-null      int64
4   col5    4 non-null      int64
5   col6    4 non-null      int32
6   col7    4 non-null      int32
7   col8    4 non-null      int32
dtypes: float32(1), int32(5), int64(2)
memory usage: 364.0+ bytes
```

- 숫자로 이루어진 문자열을 숫자형으로 변환

```
In [68]: df4 = pd.DataFrame({'id':[101,102,103],
                             'price':[2500, 3300, 4100],
                             'ratio':[0.25, 0.33, 0.41],
                             'date':['2026-01-01', '2026-01-02', '2026-01-08']})

df4
```

```
Out[68]:
```

	id	price	ratio	date
0	101	2500	0.25	2026-01-01
1	102	3300	0.33	2026-01-02
2	103	4100	0.41	2026-01-08

```
In [69]: df4.info()

<class 'pandas.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0   id      3 non-null      int64
1   price   3 non-null      int64
2   ratio   3 non-null      float64
3   date    3 non-null      str
dtypes: float64(1), int64(2), str(1)
memory usage: 228.0 bytes
```

```
In [70]: # 데이터프레임의 여러 컬럼을 동시에 변환
df4_ = df4.astype({'id':str, 'price':'int32'})
df4_.dtypes
```

```
Out[70]: id          str
        price      int32
        ratio    float64
        date      str
        dtype: object
```

```
In [73]: # 해당 컬럼의 데이터형 변환
df4['id'] = df4['id'].astype('str')
df4.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0    id      3 non-null        str
1   price    3 non-null        int64
2   ratio    3 non-null        float64
3   date     3 non-null        str
dtypes: float64(1), int64(1), str(2)
memory usage: 228.0 bytes
```

```
In [75]: # 특정 포맷을 적용해서 문자열 변환
df4['ratio_str'] = df4['ratio'].map(lambda x:f'{x:.1%}')
df4
df4.dtypes
```

```
Out[75]:
```

	id	price	ratio	date	ratio_str
0	101	2500	0.25	2026-01-01	25.0%
1	102	3300	0.33	2026-01-02	33.0%
2	103	4100	0.41	2026-01-08	41.0%

```
Out[75]: id          str
        price      int64
        ratio    float64
        date      str
        ratio_str  str
        dtype: object
```

- 날짜 문자열을 datetime64 데이터형으로 변환 : pandas.to_datetime()

```
In [78]: df4['date_str'] = pd.to_datetime(df4['date'])
df4.dtypes
```

```
Out[78]: id          str
        price      int64
        ratio    float64
        date      str
        ratio_str  str
        date_str   datetime64[us]
        dtype: object
```

```
In [80]: # 날짜를 문자열 변환
df4
df4['date_str'].dt.year
df4['date_str'].dt.month
df4['date_str'].dt.day
df4['date_str'].dt.day_of_week
df4['date_str'].dt.strftime('%d/%m/%Y')
```

```
Out[80]:
```

	id	price	ratio	date	ratio_str	date_str
0	101	2500	0.25	2026-01-01	25.0%	2026-01-01
1	102	3300	0.33	2026-01-02	33.0%	2026-01-02
2	103	4100	0.41	2026-01-08	41.0%	2026-01-08

```
Out[80]: 0    2026
         1    2026
         2    2026
         Name: date_str, dtype: int32
```

```
Out[80]: 0    1
         1    1
         2    1
         Name: date_str, dtype: int32
```

```
Out[80]: 0    1
         1    2
         2    8
         Name: date_str, dtype: int32
```

```
Out[80]: 0    3
         1    4
         2    3
         Name: date_str, dtype: int32
```

```
Out[80]: 0    01/01/2026
         1    02/01/2026
         2    08/01/2026
         Name: date_str, dtype: str
```

- 문자열 데이터에서 문자 분리/변경 등

```
In [82]: df4['date']
         df4['date'].dtype
```

```
Out[82]: 0    2026-01-01
         1    2026-01-02
         2    2026-01-08
         Name: date, dtype: str
```

```
Out[82]: <StringDtype(storage='python', na_value=nan)>
```

```
In [83]: df4['date'].str.split('-')
```

```
Out[83]: 0    [2026, 01, 01]
         1    [2026, 01, 02]
         2    [2026, 01, 08]
         Name: date, dtype: object
```

```
In [84]: df4['ratio_str']
```

```
Out[84]: 0    25.0%
         1    33.0%
         2    41.0%
         Name: ratio_str, dtype: str
```

```
In [85]: df4['ratio_str'].str.replace('%', '')
```

```
Out[85]: 0    25.0
         1    33.0
         2    41.0
         Name: ratio_str, dtype: str
```

```
In [86]: df4['ratio_str_'] = df4['ratio_str'].str.replace('%','')
df4
```

```
Out[86]:
```

	id	price	ratio	date	ratio_str	date_str	ratio_str_
0	101	2500	0.25	2026-01-01	25.0%	2026-01-01	25.0
1	102	3300	0.33	2026-01-02	33.0%	2026-01-02	33.0
2	103	4100	0.41	2026-01-08	41.0%	2026-01-08	41.0

```
In [87]: df4['ratio_str_'].astype(float)
```

```
Out[87]: 0    25.0
1    33.0
2    41.0
Name: ratio_str_, dtype: float64
```

2) 범주형(Categorical) 데이터

- Categorical은 반복되는 문자열/범주형(명목형,순서형) 데이터를 효율적으로 저장하기 위한 특수 데이터 유형
- 실무에서 메모리 절약 + 범주 순서 관리 + 그룹별연산 최적화에 매우 중요
- 일반적으로 문자열(Object/Str)은 각 행마다 문자열을 그대로 저장함
- Categorical은 내부적으로
 - 카테고리 목록(categories) : 예. 남, 여
 - 값은 정수 코드로 저장(codes) : 예. 남-> 0, 여-> 1 로 저장
 - 예. 남,여,여,남,남 -> 목록 :(남,여), 저장: 0 1 1 0 0

```
In [89]: df5 = pd.DataFrame({'Sex':['남','여','여','남','남']})
df5.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 1 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Sex      5 non-null      str
dtypes: str(1)
memory usage: 172.0 bytes
```

```
In [90]: df5['성별'] = df5.Sex.astype('category')
df5
```

```
Out[90]:
```

	Sex	성별
0	남	남
1	여	여
2	여	여
3	남	남
4	남	남

```
In [91]: df5.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Sex      5 non-null        str
1   성별      5 non-null        category
dtypes: category(1), str(1)
memory usage: 193.0 bytes
```

```
In [92]: # category의 범주(목록)
df5.성별.cat.categories
```

```
Out[92]: Index(['남', '여'], dtype='str')
```

```
In [95]: # category의 코드값
df5.성별.cat.codes
```

```
Out[95]: 0    0
1    1
2    1
3    0
4    0
dtype: int8
```

=> 실제 데이터는 정수로 저장되어 메모리 절약 효과 큼

```
In [96]: df_str = pd.DataFrame({'col':['apple', 'banana']*50000})
df_str.memory_usage(deep=True)
```

```
Out[96]: Index      132
col      5450000
dtype: int64
```

```
In [98]: df_str.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 1 columns):
#   Column  Non-Null Count  Dtype
---  -
0    col    100000 non-null  str
dtypes: str(1)
memory usage: 781.4 KB
```

```
In [99]: df_cat = df_str.copy()
df_cat['col'] = df_cat['col'].astype('category')
df_cat.memory_usage(deep=True)
```

```
Out[99]: Index      132
col      100109
dtype: int64
```

```
In [100... df_cat.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 1 columns):
#   Column  Non-Null Count  Dtype
---  -
0    col    100000 non-null  category
dtypes: category(1)
memory usage: 97.8 KB
```

- 범주형 데이터 중 순서를 고려하는 경우

```
In [101... df5['grade'] = ['A', 'B', 'A', 'C', 'B']
df5
df5.dtypes
```

```
Out[101... 
```

	Sex	성별	grade
0	남	남	A
1	여	여	B
2	여	여	A
3	남	남	C
4	남	남	B

```
Out[101... Sex          str
성별         category
grade        str
dtype: object
```

```
In [104... df5['grade'] = df5['grade'].astype('category')
df5['grade'].cat.categories
df5['grade'].cat.codes
```

```
Out[104... Index(['A', 'B', 'C'], dtype='str')
```

```
Out[104... 0    0
1    1
2    0
3    2
4    1
dtype: int8
```

```
In [107... # 순서형으로 변환
df5['grade_'] = pd.Categorical(df5['grade'],
                              categories=['C', 'B', 'A'],
                              ordered=True)

df5.dtypes
df5['grade_'].cat.ordered
```

```
Out[107... Sex          str
성별         category
grade        category
grade_       category
dtype: object
```

```
Out[107... True
```

```
In [108... # 순서가 있는 범주형 데이터 대소 비교 가능
df5['grade_'].min()
df5['grade_'].max()
```

```
Out[108... 'C'
```

```
Out[108... 'A'
```

- 카테고리 추가

```
In [109... df5['grade_'] = df5['grade_'].cat.add_categories(['D', 'E'])
df5['grade_'].cat.categories
```

```
Out[109... Index(['C', 'B', 'A', 'D', 'E'], dtype='str')
```

```
In [110...] df5['grade_']
```

```
Out[110...] 0    A
              1    B
              2    A
              3    C
              4    B
Name: grade_, dtype: category
Categories (5, str): ['C' < 'B' < 'A' < 'D' < 'E']
```

- 카테고리 제거

```
In [111...] df5['grade_'] = df5['grade_'].cat.remove_categories(['B'])
df5
```

```
Out[111...]   Sex  성별  grade  grade_
0   남   남      A      A
1   여   여      B     NaN
2   여   여      A      A
3   남   남      C      C
4   남   남      B     NaN
```

- 사용되지 않는 카테고리 제거

```
In [113...] df5['grade_'] = df5['grade_'].cat.remove_unused_categories()
df5['grade_']
```

```
Out[113...] 0    A
              1   NaN
              2    A
              3    C
              4   NaN
Name: grade_, dtype: category
Categories (2, str): ['C' < 'A']
```

- 카테고리 이름 변경

```
In [122...] # df5['grade_'] = df5['grade_'].cat.add_categories(['B'])
df5.fillna({'grade_': 'B'}, inplace=True)
df5
```

```
Out[122...]   Sex  성별  grade  grade_
0   남   남      A      A
1   여   여      B      B
2   여   여      A      A
3   남   남      C      C
4   남   남      B      B
```

Out[122...

	Sex	성별	grade	grade_
0	남	남	A	A
1	여	여	B	B
2	여	여	A	A
3	남	남	C	C
4	남	남	B	B

In [123...

```
df5['grade_'].cat.rename_categories({'A':'Excellent','B':'Good','C':'Average'})
```

Out[123...

```
0    Excellent
1         Good
2    Excellent
3     Average
4         Good
Name: grade_, dtype: category
Categories (3, str): ['Average' < 'Excellent' < 'Good']
```

- 카테고리 순서 다시 변경

In [127...

```
df5
df5['grade_'] = df5['grade_'].cat.reorder_categories(['A','B','C'])
df5['grade_']
```

Out[127...

	Sex	성별	grade	grade_
0	남	남	A	A
1	여	여	B	B
2	여	여	A	A
3	남	남	C	C
4	남	남	B	B

Out[127...

```
0    A
1    B
2    A
3    C
4    B
Name: grade_, dtype: category
Categories (3, str): ['A' < 'B' < 'C']
```

In [129...

```
df5['grade_'].cat.rename_categories({'C':'Excellent','B':'Good','A':'Average'})
```

Out[129...

```
0    Average
1         Good
2    Average
3    Excellent
4         Good
Name: grade_, dtype: category
Categories (3, str): ['Average' < 'Good' < 'Excellent']
```

- 카테고리를 새 카테고리로 완전 교체

In [132...

```
df5
df5['grade_'].cat.set_categories(['B','A','C'], ordered=False)
```


Out[132...

	Sex	성별	grade	grade_
0	남	남	A	A
1	여	여	B	B
2	여	여	A	A
3	남	남	C	C
4	남	남	B	B

Out[132...

```
0    A
1    B
2    A
3    C
4    B
Name: grade_, dtype: category
Categories (3, str): ['B', 'A', 'C']
```

- 카테고리의 ordered 속성 변경

In [133...

```
df5['grade_'].cat.as_unordered()
```

Out[133...

```
0    A
1    B
2    A
3    C
4    B
Name: grade_, dtype: category
Categories (3, str): ['A', 'B', 'C']
```

3) 수치형 데이터를 범주형 데이터로 변환

- 범주형 데이터 **Categorical 클래스** 객체로 변환
- **cut(x, bins, labels)** : 구간 경계선을 선정하여 범주형 데이터로 변환
 - x : 구간 나눌 실제 값
 - bins : 구간 경계값
 - labels: 카테고리 값
- **qcut(data, bins, label)** : 구간 경계선을 선정하지 않고 데이터 개수가 같도록 구간을 분할하여 범주형 데이터로 변환

① 구간 경계선을 선정하여 범주형 데이터로 변환 : cut()

리스트 데이터를 범주형 데이터로 변환

- 예제 데이터

In [134...

```
ages = [0.1,0.5,5,4,6,3,10,31,15,33,37,17,25,36,70,61,20,40,31,100]
len(ages)
```

Out[134...

```
20
```

- 구간 경계값, 범주 라벨 설정

In [135...

```
# 0<영유아<=5
# 5<어린이<=15
# 15<청소년<=20
```

```
# 20<청년<=40
# 40<장년<=64
# 64<노년<=100

# 구간 경계값
bins = [0, 5, 15, 20, 40, 64, 100]

# 구간별(범주) 라벨
labels = ['영유아', '어린이', '청소년', '청년', '중년', '노년']
```

- cut()로 범주형 데이터로 변경

```
In [137...] cat_ages = pd.cut(ages, bins=bins, labels=labels)
cat_ages
```

```
Out[137...] ['영유아', '영유아', '영유아', '영유아', '어린이', ..., '중년', '청소년', '청년', '청년', '노년']
Length: 20
Categories (6, str): ['영유아' < '어린이' < '청소년' < '청년' < '중년' < '노년']
```

```
In [138...] type(cat_ages)
```

```
Out[138...] pandas.Categorical
```

```
In [139...] list(cat_ages)
```

```
Out[139...] ['영유아',
'영유아',
'영유아',
'영유아',
'어린이',
'영유아',
'어린이',
'청년',
'어린이',
'청년',
'청년',
'청년',
'청소년',
'청년',
'청년',
'노년',
'중년',
'청소년',
'청년',
'청년',
'노년']
```

ages리스트와 범주형 데이터를 데이터프레임으로

```
In [142...] df_age = pd.DataFrame(data=zip(ages, cat_ages), columns=['나이', '연령대'])
df_age.head()
```

```
Out[142...]
   나이  연령대
0   0.1  영유아
1   0.5  영유아
2   5.0  영유아
3   4.0  영유아
4   6.0  어린이
```

```
In [143... df_age.info()

<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 2 columns):
#   Column   Non-Null Count  Dtype
---  -
0   나이      20 non-null    float64
1   연령대    20 non-null    str
dtypes: float64(1), str(1)
memory usage: 452.0 bytes

In [144... df_age.drop(columns = '연령대', inplace=True)
df_age['연령대'] = cat_ages
df_age.head()
df_age.dtypes
```

Out[144... 나이 연령대

0	0.1	영유아
1	0.5	영유아
2	5.0	영유아
3	4.0	영유아
4	6.0	어린이

Out[144... 나이 float64
 연령대 category
 dtype: object

```
In [145... df_age.info()

<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 2 columns):
#   Column   Non-Null Count  Dtype
---  -
0   나이      20 non-null    float64
1   연령대    20 non-null    category
dtypes: category(1), float64(1)
memory usage: 532.0 bytes
```

```
In [147... df_age.describe()
```

Out[147... 나이

count	20.000000
mean	27.230000
std	25.948128
min	0.100000
25%	5.750000
50%	22.500000
75%	36.250000
max	100.000000

```
In [149... df_age.연령대.value_counts()
df_age.연령대.value_counts().sort_index()
```

```
Out[149...] 연령대
청년      7
영유아     5
어린이     3
청소년     2
노년       2
중년       1
Name: count, dtype: int64
```

```
Out[149...] 연령대
영유아     5
어린이     3
청소년     2
청년       7
중년       1
노년       2
Name: count, dtype: int64
```

참고: Categorical 클래스 객체

- 카테고리명 속성 : Categorical.categories
- 코드 속성 : Categorical.codes
 - 인코딩한 카테고리 값을 정수로 받음

```
In [150...] type(cat_ages)
```

```
Out[150...] pandas.Categorical
```

```
In [152...] cat_ages.categories
cat_ages.codes
```

```
Out[152...] Index(['영유아', '어린이', '청소년', '청년', '중년', '노년'], dtype='str')
```

```
Out[152...] array([0, 0, 0, 0, 1, 0, 1, 3, 1, 3, 3, 2, 3, 3, 5, 4, 2, 3, 3, 5],
      dtype=int8)
```

```
In [153...] df_age.연령대.cat.categories
df_age.연령대.cat.codes
```

```
Out[153...] Index(['영유아', '어린이', '청소년', '청년', '중년', '노년'], dtype='str')
```

```
Out[153...] 0      0
1      0
2      0
3      0
4      1
5      0
6      1
7      3
8      1
9      3
10     3
11     2
12     3
13     3
14     5
15     4
16     2
17     3
18     3
19     5
dtype: int8
```

② 데이터 개수가 같도록 데이터 분할 : qcut()

: 구간 경계선을 지정하지 않고 데이터의 사분위수(quantile) 기준으로 분할

- 형식 : `pd.qcut(data, 구간수, labels=[d1, d2, ...])`
- 예. 1000개의 데이터를 4구간으로 나누려고 한다면
 - qcut 명령어를 사용 한 구간마다 250개씩 나누게 된다.
 - 예외) 같은 숫자인 경우에는 같은 구간으로 처리한다.
- 예제 데이터

```
In [154...] np.random.seed(2)
data = np.random.randint(20, size=20)
data
```

```
Out[154...] array([ 8, 15, 13,  8, 11, 18, 11,  8,  7,  2, 17, 11, 15,  5,  7,  3,  6,
        4, 10, 11], dtype=int32)
```

- 20개의 데이터를 동일한 개수를 갖는 4개 구간으로 나누어 범주형 데이터로 생성하고, 각 구간의 label은 Q1, Q2, Q3, Q4 로 설정

```
In [155...] qcut_data = pd.qcut(data, 4, labels=['Q1', 'Q2', 'Q3', 'Q4'])
qcut_data
```

```
Out[155...] ['Q2', 'Q4', 'Q4', 'Q2', 'Q3', ..., 'Q1', 'Q1', 'Q1', 'Q3', 'Q3']
Length: 20
Categories (4, str): ['Q1' < 'Q2' < 'Q3' < 'Q4']
```

```
In [156...] type(qcut_data)
```

```
Out[156...] pandas.Categorical
```

```
In [158...] df_qcut = pd.DataFrame(data, columns=['data'])
df_qcut['qcut'] = qcut_data
df_qcut.head()
```

```
Out[158...]
   data  qcut
0     8   Q2
1    15   Q4
2    13   Q4
3     8   Q2
4    11   Q3
```

```
In [160...] df_qcut['qcut'].value_counts()
```

```
Out[160...] qcut
Q1     5
Q2     5
Q3     5
Q4     5
Name: count, dtype: int64
```

```
In [161... data
np.sort(data)
```

```
Out[161... array([ 8, 15, 13,  8, 11, 18, 11,  8,  7,  2, 17, 11, 15,  5,  7,  3,  6,
        4, 10, 11], dtype=int32)
```

```
Out[161... array([ 2,  3,  4,  5,  6,  7,  7,  8,  8,  8, 10, 11, 11, 11, 11, 13, 15,
        15, 17, 18], dtype=int32)
```

```
In [162... qcut_data.codes
```

```
Out[162... array([1, 3, 3, 1, 2, 3, 2, 1, 1, 0, 3, 2, 3, 0, 1, 0, 0, 0, 2, 2],
        dtype=int8)
```

```
In [163... qcut_data.categories
```

```
Out[163... Index(['Q1', 'Q2', 'Q3', 'Q4'], dtype='str')
```

```
In [165... df_qcut['qcut'].cat.categories
df_qcut['qcut'].cat.codes
```

```
Out[165... Index(['Q1', 'Q2', 'Q3', 'Q4'], dtype='str')
```

```
Out[165... 0      1
1      3
2      3
3      1
4      2
5      3
6      2
7      1
8      1
9      0
10     3
11     2
12     3
13     0
14     1
15     0
16     0
17     0
18     2
19     2
dtype: int8
```

cut와 qcut 비교

```
In [167... # cut(bins=구간수) => 등간격으로 지정한 구간수로 분할
sample = [1,2,3,4,5,100]
sam_cut = pd.cut(sample, bins=3, labels=['c1','c2','c3'])
sam_cut.value_counts()
# 세 구간 (0.901, 34.0], (34.0, 67.0], (67.0, 100.0] 으로 분할 (등간격)
```

```
Out[167... c1      5
c2      0
c3      1
Name: count, dtype: int64
```

```
In [168... sam_cut.categories
```

```
Out[168... Index(['c1', 'c2', 'c3'], dtype='str')
```

```
In [169... sam_qcut = pd.qcut(sample, q=3, labels=['c1','c2','c3'])
sam_qcut
```

```
Out[169... ['c1', 'c1', 'c2', 'c2', 'c3', 'c3']
Categories (3, str): ['c1' < 'c2' < 'c3']
```

```
In [170... sam_qcut.value_counts()
```

```
Out[170... c1    2
c2    2
c3    2
Name: count, dtype: int64
```

=> 데이터 분포가 비대칭인 경우 cut보다 qcut으로 균등하게 분할하는 것을 활용

실무에서 활용

- cut() 사용 : 범주화 기준이 명확할 때
 - 정책/비즈니스 룰 기반
- qcut() 사용 : 상대적 등급화
 - 고객 세그멘테이션 : RFM
 - 상위 20%, 하위 20% 분류

학습내용

4. 데이터 삭제

- drop()

5. 결측치 처리

- 결측치 확인 : isna(), isnull()
- 결측치 삭제 : dropna()
- 결측치 대체 : fillna()

6. 데이터 변경

- 데이터 자료형 변경 : astype()
 - 수치형 데이터 범주화 : cut(), qcut()
-